AI-generated code

Finding errors in code

D5 Errors typical of AI-generated code

D5-L1 Invented functions, parameters and syntax

Look up every function, parameter and statement for your exact version, and run it on small data before it reaches a pipeline.

CREATE ROW ACCESS POLICY	Snowflake syntax. Databricks: SQL UDF plus ALTER TABLE ... SET ROW FILTER.
F.col('a').isnotnull()	Unknown attribute becomes a struct field lookup, then 'Column' object is not callable. The method is isNotNull.
Three ways to confirm an API exists	Official docs for your version, hasattr or help() in a notebook, inspect.signature for parameters.
The costly invented-API case	Code that runs: a real parameter with a different meaning, or a function from a newer version.

D5-L2 Deprecated APIs and version mismatches

Assume suggested code targets some other version. Check deprecations for yours, turn warnings into test failures, and compare versions numerically.

Chained assignment in pandas 3	df['a'][i] = x never updates df (Copy-on-Write). Use df.loc[i, 'a'] = x.
pandas 3 removals	DataFrame.append (use pd.concat) and applymap (use DataFrame.map).
input_file_name() on DBR 17.3 LTS	Not supported. Use _metadata.file_path.
Comparing version strings	Never as text: '3.10' < '3.9'. Compare integer tuples or use packaging.version.

D5-L3 Five review checks for AI-written code

Review AI code for edge cases, self-confirming tests, secrets, needless structure and, in DAX, filters removed too widely.

```
def test_share():
    assert share(3, 4) == 3 / 4   # repeats the code
```

Five review checks for AI code	Edge cases, self-confirming tests, secrets, over-engineering, DAX filter context.
Self-confirming test	Expected value computed with the code's own formula. Use hand-worked values instead.
Where secrets live	Secret scopes or Key Vault, read at run time. Never in code or prompts.
ALL(fact_table) in a share measure	Removes every filter, including wave and region slicers. Use ALL on the one dimension you mean.

D6 Verifying AI output

D6-L1 Read before you run, and check each API

Restate the goal, list and check every name, hunt for side effects, then run on development data.

Five steps of the read-through	Restate the goal, list names, check each name, find side effects, run on dev data.
Side effects to hunt for	mode('overwrite'), DROP, DELETE, TRUNCATE, VACUUM, writes to production names.
Quick existence checks in a notebook	hasattr for names, inspect.signature(...).parameters for parameters, DESCRIBE TABLE or df.columns for columns.
What a read-through does not do	Prove correctness. It clears visible mistakes so tests can find the rest.

D6-L2 Small data with a known answer, failing test first

Hand-built data, hand-worked answers, a test that fails first, and a deliberate break to prove the test bites.

Known-answer data	5 to 10 hand-built rows: normal case, duplicate, null, zero total, empty group, boundary.
Test-first order	Write the test, watch it fail, accept the code, watch it pass, break the code on purpose, watch it fail.
assertDataFrameEqual defaults (PySpark 4.0)	checkRowOrder=False, rtol=1e-05, atol=1e-08.
Why break code on purpose	If the test still passes, it misses that rule. A quick form of mutation testing.

D6-L3 Ask for assumptions, then compare a second approach

Make the assistant state its assumptions, check them on real data, keep the important ones as checks, and confirm results by an independent route.

Prompt for assumptions	'List every assumption this code makes about its input data, one per line.' Then check each on real data.
Common data assumptions to check	Unique keys per grain, non-null positive weights, id formats, UTC timestamps, one lookup row per key.
An independent second approach	SQL vs DataFrame code, pandas vs Spark on an extract, gold totals vs silver aggregates. Not the same assistant rewriting itself.
When two methods disagree	Run both on known-answer data. The one matching the hand-worked answer is right.

D8 Prompting for correct code

D8-L1 State versions and constraints in every prompt

Every coding prompt names versions, platform and dialect, the constraints from your standard, and what not to do.

```
Environment: Databricks Runtime 17.3 LTS (Spark 4.0, Python 3.12),
PySpark DataFrame API, Delta tables in Unity Catalog.
```

```
Local tests: pandas 3.0, pytest 9.
```

Four things every coding prompt states	Versions and platform, the dialect, the relevant standards, and what not to do.
Why name the SQL dialect	Syntax differs by platform. 'SQL' alone invites Snowflake or T-SQL statements.
Where to keep the prompt header	In the repository next to CLAUDE.md, updated when versions change.

D8-L2 State expected outputs and tests in the prompt

Give a worked example, the edge cases with answers, the output format and the tests. Then review against them.

```
Input (respondent_id, reach_flag, final_weight):
  R1, 1, 2.0   R2, 0, 1.0   R3, 1, 1.0   R4, null, 1.0
Expected: weighted_reach = 3.0, weighted_base = 4.0, pct = 0.75
R4 has no reach answer: exclude it from the base.
```

Four things that specify correct code in a prompt	A worked example, edge cases with answers, the output format, and fixed tests.
Instruction that protects your tests	'Make these tests pass without editing them. If a test seems wrong, say so.'
Why work the example out yourself	If you cannot compute the answer by hand, the rule is not clear enough to ask for code.

D8-L3 Iterate on a failing answer

Send the failing test, the error and the smallest input. Fix one thing per round, diff the tests, and stop after two failed rounds.

```
test_zero_share fails:
  assert compare_waves({"a": 0.0}, {"a": 0.12}) == ["a"]
  got [] instead of ["a"]
Line 6 uses `if current_pct and prior_pct:`. I think 0.0 is treated as missing.
Fix only this. Do not change the tests or other behaviour.
```

What to send when AI code fails a test	The failing assertion, actual vs expected, the smallest input, what you checked, and 'fix only this'.
The test-tampering check	Diff the test file after every answer. Removed or changed assertions need your approval.
When to stop iterating	After two focused rounds without progress: debug it yourself, or restart in a fresh conversation.